

For a **1-hour PostgreSQL Fundamentals session**, you can structure the content as follows:

PostgreSQL Fundamentals (60 Minutes)

1. Introduction to PostgreSQL (5 mins)

- What is PostgreSQL?
- Relational Database Concepts
- Database vs Schema vs Table
- PostgreSQL Use Cases

2. DDL – Data Definition Language (15 mins)

Objective: Learn how to create and modify database structures.

Topics:

- CREATE DATABASE
- CREATE TABLE
- Data Types (INTEGER, VARCHAR, TEXT, DATE, BOOLEAN, TIMESTAMP)
- ALTER TABLE
- DROP TABLE
- TRUNCATE TABLE

Example:

```
CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    salary NUMERIC(10,2),
```

```
        joining_date DATE  
    );
```

3. DML – Data Manipulation Language (15 mins)

Objective: Learn how to manage data.

Topics:

- INSERT
- UPDATE
- DELETE
- SELECT
- WHERE clause
- ORDER BY
- LIMIT

Example:

```
INSERT INTO employees(name, salary)  
VALUES ('John', 50000);
```

```
UPDATE employees  
SET salary = 55000  
WHERE id = 1;
```

```
DELETE FROM employees  
WHERE id = 1;
```

4. Integrity Constraints (10 mins)

Objective: Ensure data quality and consistency.

Topics:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT

Example:

```
CREATE TABLE departments (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    department_id INT,  
    FOREIGN KEY (department_id)  
    REFERENCES departments(id)  
);
```

5. ACID Properties (10 mins)

Objective: Understand transaction management.

Topics:

- Atomicity

- Consistency
- Isolation
- Durability
- COMMIT
- ROLLBACK

Example:

```
BEGIN;
```

```
UPDATE accounts  
SET balance = balance - 1000  
WHERE id = 1;
```

```
UPDATE accounts  
SET balance = balance + 1000  
WHERE id = 2;
```

```
COMMIT;
```

Rollback Example:

```
BEGIN;
```

```
UPDATE accounts  
SET balance = balance - 1000  
WHERE id = 1;
```

```
ROLLBACK;
```

6. Quick Hands-on Exercise (5 mins)

Build:

- `students` table
- Insert 3 records
- Update one record
- Delete one record
- Demonstrate a transaction with COMMIT/ROLLBACK

Learning Outcomes

By the end of this session, participants will be able to:

- Understand PostgreSQL architecture and terminology.
- Create and modify tables using DDL commands.
- Perform CRUD operations using DML.
- Apply integrity constraints for data validation.
- Use transactions and understand ACID properties.
- Write basic PostgreSQL queries confidently.

PostgreSQL Data Models (1 Hour)

Learning Objective: Understand different data modeling approaches and how they are implemented in PostgreSQL.

1. Introduction to Data Modeling (5 mins)

Topics

- What is a Data Model?
- Why Data Modeling Matters
- Conceptual vs Logical vs Physical Models
- Choosing the right model for the right workload

Example Use Cases

- Banking → Normalized Model
- Social Media → Graph Model
- Data Warehouse → Star Schema
- Reporting → Views

2. Relational Data Modeling & Normal Forms (20 mins)

Objective

Design efficient transactional databases.

Topics

- Entities and Relationships
- Primary Keys
- Foreign Keys
- Normalization Overview

Normal Forms

First Normal Form (1NF)

- No repeating groups
- Atomic values

Second Normal Form (2NF)

- Remove partial dependency

Third Normal Form (3NF)

- Remove transitive dependency

BCNF (Brief Introduction)

PostgreSQL Example

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    customer_name VARCHAR(100)  
);
```

```
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES customers(customer_id),  
    order_date DATE  
);
```

Benefits

- Reduced redundancy
- Improved consistency
- Easier updates

3. Network / Graph Data Modeling (10 mins)

Objective

Represent complex relationships.

Use Cases

- Social Networks
- Fraud Detection
- Recommendation Systems
- Supply Chain Networks

PostgreSQL Approach

Represent nodes and edges using tables.

```
CREATE TABLE users (  
    user_id SERIAL PRIMARY KEY,  
    name VARCHAR(100)  
);
```

```
CREATE TABLE friendships (  
    user1_id INT,  
    user2_id INT,  
    PRIMARY KEY(user1_id, user2_id)  
);
```

Recursive Query Example

```
WITH RECURSIVE network AS (  
    SELECT user1_id, user2_id
```



```
        FROM friendships
    )
SELECT * FROM network;
```

PostgreSQL Features

- Recursive CTEs
- JSONB relationships
- Graph extensions (Apache AGE)

4. Dimensional Modeling (Star Schema) (15 mins)

Objective

Build analytics and reporting systems.

Concepts

Fact Table

Stores measurable events.

Examples:

- Sales
- Transactions
- Orders

Dimension Table

Stores descriptive information.

Examples:

- Customer
- Product
- Date
- Location

Star Schema Example

```
CREATE TABLE dim_customer (  
    customer_key SERIAL PRIMARY KEY,  
    customer_name VARCHAR(100)  
);
```

```
CREATE TABLE dim_product (  
    product_key SERIAL PRIMARY KEY,  
    product_name VARCHAR(100)  
);
```

```
CREATE TABLE fact_sales (  
    sales_id SERIAL PRIMARY KEY,  
    customer_key INT,  
    product_key INT,  
    sales_amount NUMERIC(10,2)  
);
```

Advantages

- Fast reporting
- Easy Power BI integration
- Simple aggregations

Real-world Example

- Retail Sales Dashboard
- BFSI Transaction Analytics
- Healthcare Claims Reporting

5. PostgreSQL Views (5 mins)

Objective

Simplify reporting and security.

Topics

- Simple Views
- Materialized Views
- Reporting Layer

View Example

```
CREATE VIEW customer_sales AS
SELECT
    c.customer_name,
    SUM(f.sales_amount) total_sales
FROM fact_sales f
JOIN dim_customer c
ON c.customer_key = f.customer_key
GROUP BY c.customer_name;
```

Materialized View

```
CREATE MATERIALIZED VIEW monthly_sales AS
```

```
SELECT
    DATE_TRUNC( 'month', order_date) month,
    SUM(sales_amount) revenue
FROM sales
GROUP BY month;
Refresh:

REFRESH MATERIALIZED VIEW monthly_sales;
```

6. Model Selection Guide (5 mins)

Model	Best For	Example
Normalized (3NF)	OLTP Systems	Banking, ERP
Graph/Network	Relationship Analysis	Social Media, Fraud Detection
Star Schema	Analytics & BI	Power BI, Data Warehouses
Views	Reporting & Security	Dashboards, Business Reports

PostgreSQL Joins & Subqueries (1 Hour)

Learning Objective: Learn how to combine, filter, and retrieve data from multiple tables using joins and subqueries.

1. Introduction to Joins (5 mins)

Why Joins?

- Data is usually spread across multiple tables.
- Joins help combine related information.
- Based on Primary Key and Foreign Key relationships.

Sample Tables

customers

customer_

r_id name

1 John

2 Alice

orders

order customer amo

_id r_id unt

101 1 500

102 1 700

103 2 300

2. INNER JOIN (10 mins)

Objective

Retrieve matching records from both tables.

SELECT

 c.customer_name,

 o.order_id,

 o.amount

FROM customers c

```
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

Result

Only customers having orders.

Use Cases

- Customer Orders
- Employee Departments
- Student Courses

3. LEFT JOIN (10 mins)

Objective

Return all records from the left table and matching records from the right table.

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Result

- Includes customers with no orders.
- Missing matches appear as NULL.

Common Use Cases

- Customers without orders

- Employees without projects

4. RIGHT JOIN & FULL JOIN (10 mins)

RIGHT JOIN

Returns all records from the right table.

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

FULL JOIN

Returns all records from both tables.

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
FULL JOIN orders o
ON c.customer_id = o.customer_id;
```

Use Cases

- Data Reconciliation
- Migration Validation
- Audit Reporting

5. Introduction to Subqueries (5 mins)

What is a Subquery?

A query inside another query.

```
SELECT *  
FROM orders  
WHERE amount >  
(  
    SELECT AVG(amount)  
    FROM orders  
);
```

Result

Orders greater than average order value.

6. Non-Correlated Subqueries (7 mins)

Objective

Inner query executes once.

```
SELECT customer_name  
FROM customers  
WHERE customer_id IN  
(  
    SELECT customer_id  
    FROM orders  
);
```

Characteristics

- Runs independently.

- Faster and easier to optimize.
- Most commonly used.

Examples

- IN
- EXISTS
- Scalar Subqueries

7. Correlated Subqueries (8 mins)

Objective

Inner query depends on the outer query.

```
SELECT
    c.customer_name
FROM customers c
WHERE EXISTS
(
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Example: Customer Total Orders

```
SELECT
    customer_name
FROM customers c
WHERE (
    SELECT COUNT(*)
```

```
FROM orders o
WHERE o.customer_id = c.customer_id
) > 2;
```

Characteristics

- Executes for each outer row.
- More powerful but can be slower.

8. Join vs Subquery (3 mins)

Feature	Join	Subquery
Performance	Usually Faster	Can be Slower
Readability	Better for Multiple Tables	Better for Filtering
Reporting	Excellent	Moderate
Aggregation	Easy	Good

Best Practice

- Use JOINS for combining data.
- Use Subqueries for filtering and calculations.
- Consider CTEs for complex logic.

Hands-On Exercise (2 mins)

Create:

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    customer_name VARCHAR(100)  
);
```

```
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INT,  
    amount NUMERIC(10,2)  
);
```

Practice:

- 1 INNER JOIN customers and orders.
- 2 LEFT JOIN to find customers without orders.
- 3 Find orders above average amount using a subquery.
- 4 Use a correlated subquery to find customers with more than 2 orders.

PostgreSQL CTEs & Functions (1 Hour)

Learning Objective: Learn advanced query structures using Common Table Expressions (CTEs) and PostgreSQL Functions.

1. Introduction to CTEs (5 mins)

What is a CTE?

A Common Table Expression (CTE) is a temporary result set that exists only during query execution.

Benefits

- Improves readability
- Simplifies complex queries
- Enables recursion
- Easier debugging and maintenance

Basic Syntax

```
WITH sales_summary AS (  
    SELECT customer_id,  
           SUM(amount) AS total_sales  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT *  
FROM sales_summary;
```

2. Non-Recursive CTEs (15 mins)

Objective

Break large queries into manageable parts.

Example: Customer Sales Summary

```
WITH customer_sales AS (  
    SELECT  
        customer_id,  
        SUM(amount) total_amount  
    FROM orders  
    GROUP BY customer_id
```

```
)  
SELECT *  
FROM customer_sales  
WHERE total_amount > 1000;
```

Multiple CTEs

```
WITH customer_sales AS (  
    SELECT customer_id,  
           SUM(amount) total_sales  
    FROM orders  
    GROUP BY customer_id  
) ,  
top_customers AS (  
    SELECT *  
    FROM customer_sales  
    WHERE total_sales > 5000  
)  
SELECT *  
FROM top_customers;
```

Use Cases

- Reporting Queries
- Data Cleaning
- ETL Transformations

3. Recursive CTEs (15 mins)

Objective

Handle hierarchical and tree-structured data.

Example Data

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    manager_id INT  
);
```

Recursive Query

```
WITH RECURSIVE employee_hierarchy AS (  
    SELECT  
        emp_id,  
        emp_name,  
        manager_id,  
        1 AS level  
    FROM employees  
    WHERE manager_id IS NULL  
  
    UNION ALL  
  
    SELECT  
        e.emp_id,  
        e.emp_name,  
        e.manager_id,  
        h.level + 1  
    FROM employees e  
    JOIN employee_hierarchy h  
    ON e.manager_id = h.emp_id  
)  
SELECT *  
FROM employee_hierarchy;
```


Use Cases

- Employee Hierarchies
- Category Trees
- Organization Charts
- Bill of Materials

4. PostgreSQL Functions Overview (5 mins)

Why Functions?

- Reusable logic
- Better maintainability
- Centralized business rules
- Improved productivity

Types of Functions

- 1 Scalar Functions
- 2 Aggregate Functions
- 3 Table-Valued Functions

5. Scalar Functions (8 mins)

Objective

Return a single value.

Example

```
CREATE OR REPLACE FUNCTION calculate_tax(  
    amount NUMERIC  
)  
RETURNS NUMERIC  
AS $$  
BEGIN  
    RETURN amount * 0.18;  
END;  
$$ LANGUAGE plpgsql;
```

Usage

```
SELECT calculate_tax(1000);
```

Common Use Cases

- Tax Calculation
- Discounts
- Business Rules

6. Aggregate Functions (5 mins)

Built-in Aggregate Functions

```
SELECT  
    COUNT(*) ,  
    SUM(amount) ,  
    AVG(amount) ,  
    MAX(amount) ,  
    MIN(amount)  
FROM orders;
```


Example Reporting Query

```
SELECT
    customer_id,
    SUM(amount) total_sales
FROM orders
GROUP BY customer_id;
```

Use Cases

- Dashboards
- Analytics
- KPI Calculations

7. Table-Valued Functions (5 mins)

Objective

Return multiple rows.

Example

```
CREATE OR REPLACE FUNCTION get_customer_orders(
    p_customer_id INT
)
RETURNS TABLE (
    order_id INT,
    amount NUMERIC
)
AS $$
BEGIN
    RETURN QUERY
    SELECT
```

```
        o.order_id,
        o.amount
    FROM orders o
    WHERE o.customer_id = p_customer_id;
END;
$$ LANGUAGE plpgsql;
```

Usage

```
SELECT *
FROM get_customer_orders(1);
```

Use Cases

- Reporting APIs
- Data Access Layers
- Reusable Query Logic

8. CTEs vs Functions (2 mins)

Feature	CTE	Function
Scope	Single Query	Reusable
Performance	Query Specific	Reusable Logic
Parameters	No	Yes
Recursion	Yes	No
Business Logic	Limited	Excellent

Hands-On Exercise (5 mins)

Create an Orders Table

```
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INT,  
    amount NUMERIC(10,2)  
);
```

Practice

- 1 Create a non-recursive CTE to calculate customer sales.
- 2 Create a recursive CTE for employee hierarchy.
- 3 Create a scalar function for GST calculation.
- 4 Use aggregate functions to generate KPIs.
- 5 Create a table-valued function to fetch customer orders.

PostgreSQL CASE Statements & Conditional Logic (1 Hour)

Learning Objective: Learn how to implement business rules and decision-making directly within SQL queries using CASE expressions.

1. Introduction to Conditional Logic (5 mins)

Why Conditional Logic?

Real-world databases often require:

- Categorizing data
- Applying business rules

- Creating calculated columns
- Generating reports with custom labels

Examples

- Customer Segmentation
- Grade Calculation
- Salary Bands
- Order Status Classification

2. Understanding CASE Statements (10 mins)

Basic Syntax

```
SELECT
    CASE
        WHEN condition1 THEN result1
        WHEN condition2 THEN result2
        ELSE result3
    END;
```

Example

```
SELECT
    order_id,
    amount,
    CASE
        WHEN amount > 5000 THEN 'High Value'
        ELSE 'Regular'
    END AS order_type
FROM orders;
```

Output

order_id	amount	order_type
101	8000	High Value
102	3000	Regular

3. Multiple Conditions (10 mins)

Customer Classification

```
SELECT
    customer_id,
    total_sales,
    CASE
        WHEN total_sales >= 10000 THEN 'Platinum'
        WHEN total_sales >= 5000 THEN 'Gold'
        WHEN total_sales >= 2000 THEN 'Silver'
        ELSE 'Bronze'
    END AS customer_tier
FROM customer_summary;
```

Benefits

- Simplifies reporting
- Implements business logic
- Easy to maintain

4. CASE in SELECT Clause (10 mins)

Salary Band Example

```
SELECT
    employee_name,
    salary,
    CASE
        WHEN salary < 30000 THEN 'Junior'
        WHEN salary BETWEEN 30000 AND 70000 THEN 'Mid-Level'
        ELSE 'Senior'
    END AS employee_level
FROM employees;
```

Use Cases

- Employee Grading
- Product Categories
- Risk Assessment

5. CASE in WHERE Clause (5 mins)

Dynamic Filtering

```
SELECT *
FROM orders
WHERE
CASE
    WHEN amount > 5000 THEN TRUE
    ELSE FALSE
END;
```

Alternative (Preferred)

```
SELECT *
```



```
FROM orders
WHERE amount > 5000;
```

Discussion

- CASE works in WHERE clauses.
- Simple conditions are usually clearer.

6. CASE in ORDER BY Clause (5 mins)

Custom Sorting

```
SELECT *
FROM orders
ORDER BY
CASE
    WHEN status = 'Critical' THEN 1
    WHEN status = 'Pending' THEN 2
    ELSE 3
END;
```

Use Cases

- Priority Queues
- Ticketing Systems
- Workflow Management

7. CASE with Aggregations (10 mins)

Conditional Counting

```
SELECT
```

```
        COUNT(  
            CASE  
                WHEN amount > 5000 THEN 1  
            END  
        ) AS high_value_orders  
FROM orders;
```

Conditional Summation

```
SELECT  
    SUM(  
        CASE  
            WHEN status = 'Completed'  
            THEN amount  
            ELSE 0  
        END  
    ) AS completed_sales  
FROM orders;
```

KPI Dashboard Example

```
SELECT  
    SUM(CASE WHEN status='Completed' THEN amount ELSE 0 END) completed_sales,  
    SUM(CASE WHEN status='Pending' THEN amount ELSE 0 END) pending_sales,  
    COUNT(CASE WHEN amount > 5000 THEN 1 END) premium_orders  
FROM orders;
```

8. CASE vs COALESCE vs NULLIF (3 mins)

CASE

```
CASE  
    WHEN salary IS NULL THEN 0  
    ELSE salary  
END
```


COALESCE

```
COALESCE(salary, 0)
```

NULLIF

```
NULLIF(quantity, 0)
```

Best Practice

- Use CASE for complex conditions.
- Use COALESCE for NULL replacement.
- Use NULLIF to avoid divide-by-zero errors.

Hands-On Exercise (2 mins)

Create:

```
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    amount NUMERIC(10,2),  
    status VARCHAR(20)  
);
```

Practice

- 1 Categorize orders as Small, Medium, Large.
- 2 Create customer tiers using CASE.
- 3 Generate completed and pending sales totals.
- 4 Sort records using custom priority logic.
- 5 Replace NULL values using CASE and COALESCE.

- **PostgreSQL Window Functions (1 Hour)**
- **Learning Objective:** Learn how to perform advanced analytics using Window Functions without collapsing rows like traditional aggregate functions.
-
- **1. Introduction to Window Functions (5 mins)**
- **What are Window Functions?**
- Window Functions perform calculations across a set of rows while preserving individual row details.
- **Benefits**
- Ranking data
- Finding previous/next values
- Running totals
- Trend analysis
- Time-series analytics
- **Syntax**
- `function_name()`
- `OVER (`
- `PARTITION BY column`
- `ORDER BY column`
- `)`
- **Sample Data**

- CREATE TABLE sales (
- sale_id SERIAL PRIMARY KEY,
- employee_name VARCHAR(100),
- sales_month DATE,
- sales_amount NUMERIC(10,2)
-);

•

- **2. ROW_NUMBER() (10 mins)**

- **Objective**

- Assign a unique sequential number to each row.

- SELECT
- employee_name,
- sales_amount,
- ROW_NUMBER() OVER (
- ORDER BY sales_amount DESC
-) AS row_num
- FROM sales;
-

- **Use Cases**

- Pagination

- Top N Records

- Deduplication

- **Latest Record Per Customer**

- SELECT *
 - FROM (- SELECT *,
 - ROW_NUMBER() OVER (- PARTITION BY customer_id
 - ORDER BY order_date DESC
 -) rn
 - FROM orders
 -) t
 - WHERE rn = 1;

-

- **3. RANK() (10 mins)**

- **Objective**

- Assign rankings while allowing gaps.

- SELECT
 - employee_name,
 - sales_amount,
 - RANK() OVER (- ORDER BY sales_amount DESC
 -) sales_rank
 - FROM sales;

- **Output**

- Notice Rank 3 is skipped.
- **Use Cases**
- Competition Rankings
- Leaderboards
- Sales Performance
-
- **4. DENSE_RANK() (10 mins)**
- **Objective**
- Assign rankings without gaps.
- `SELECT`
- `employee_name,`
- `sales_amount,`
- `DENSE_RANK() OVER (`
- `ORDER BY sales_amount DESC`
- `) sales_rank`
- `FROM sales;`
- **Output**

•

- **5. LEAD() (10 mins)**
- **Objective**
- Access the next row value.
- `SELECT`
- `sales_month,`
- `sales_amount,`
- `LEAD(sales_amount)`
- `OVER (`
- `ORDER BY sales_month`
- `) AS next_month_sales`
- `FROM sales;`
- **Output**

•

NULL

- **Sales Growth Analysis**
- `SELECT`
- `sales_month,`
- `sales_amount,`
- `LEAD(sales_amount)`
- `OVER (ORDER BY sales_month) -`
- `sales_amount AS growth`
- `FROM sales;`

- **Use Cases**
- Forecasting
- Growth Tracking
- Time Series Analysis

-

- **6. LAG() (10 mins)**

- **Objective**

- Access the previous row value.

- SELECT
- sales_month,
- sales_amount,
- LAG(sales_amount)
- OVER (
- ORDER BY sales_month
-) AS previous_sales
- FROM sales;

- **Month-over-Month Growth**

- SELECT
- sales_month,
- sales_amount,
- sales_amount -
- LAG(sales_amount)
- OVER (ORDER BY sales_month)
- AS monthly_growth
- FROM sales;

- **Use Cases**

- Trend Analysis
- Financial Reporting
- Customer Behavior Analysis

•

- **7. Combining Window Functions (3 mins)**

- `SELECT`
- `employee_name,`
- `sales_amount,`
- `ROW_NUMBER() OVER(ORDER BY sales_amount DESC),`
- `RANK() OVER(ORDER BY sales_amount DESC),`
- `DENSE_RANK() OVER(ORDER BY sales_amount DESC)`
- `FROM sales;`
- **Compare Ranking Functions**

•

- **Hands-On Exercise (2 mins)**

- Create:
- `CREATE TABLE sales (`
- `employee_name VARCHAR(100),`
- `sales_month DATE,`
- `sales_amount NUMERIC(10,2)`
- `);`

- Practice:
- Assign row numbers to all sales records.
- Rank employees by sales.
- Compare RANK() and DENSE_RANK().
- Find previous month's sales using LAG().
- Find next month's sales using LEAD().
- Calculate month-over-month growth.